

Data Profiler

Data Preprocessing and Feature Engineering

Theory Concepts & Definitions

Prepared as part of the PR.1 Data Profiler project — covering the theory behind data analysis, tensors, data acquisition, cleaning, exploratory data analysis, data profiling, and feature engineering, in support of the accompanying Jupyter Notebook implementation.

"Quality is our Motto."

1. What is Data Analysis?

Data Analysis is the systematic process of inspecting, cleaning, transforming, and modeling data in order to discover useful information, draw sound conclusions, and support decision-making. It combines statistics, domain expertise, and computational tools to turn raw data into actionable insight.

The Four Types of Data Analysis

Descriptive	Summarizes <i>what happened</i> using statistics and visualizations (e.g., average customer spend last quarter).
Diagnostic	Investigates <i>why</i> something happened by exploring relationships between variables (e.g., why churn spiked in March).
Predictive	Uses historical data and statistical/ML models to forecast <i>what is likely to happen</i> (e.g., which customers will churn next).
Prescriptive	Recommends <i>what action to take</i> based on predictions (e.g., offer a retention discount to at-risk customers).

This project applies descriptive analysis (summary statistics), diagnostic analysis (bivariate/multivariate EDA), and lays the groundwork for predictive analysis by framing and preparing data for a churn-prediction model.

2. Planning a Data Science Project

A Data Science project is rarely a single, linear pass — it is an iterative lifecycle. The steps below are based on the widely used **CRISP-DM** (Cross-Industry Standard Process for Data Mining) framework, adapted for a typical analytics project:

- 1. Business Understanding** — Clarify the problem and define a measurable success metric.
- 2. Data Understanding** — Identify available data sources and assess data quality at a glance.
- 3. Data Acquisition** — Collect and import data from every source (files, databases, APIs) into one place.
- 4. Data Cleaning & Preprocessing** — Handle missing values, fix inconsistent types, remove duplicates and outliers.
- 5. Exploratory Data Analysis (EDA)** — Understand distributions, relationships, and patterns using statistics and visualization.
- 6. Feature Engineering** — Create, transform, encode, and scale features so a model can learn from them effectively.
- 7. Modeling** — Select and train candidate machine learning algorithms.

8. Evaluation — Validate model performance against the business goal using appropriate metrics.

9. Deployment & Monitoring — Ship the model into production and monitor for performance or data drift over time.

10. Communication — Report findings, visuals, and recommendations back to stakeholders.

The Data Profiler project focuses on steps 2 through 6 — building a solid, clean, well-understood dataset before any model is trained.

3. Framing a Machine Learning Problem

Framing means translating a vague business question into a precise, solvable machine learning task. For this project:

Business question	“Which customers are at risk of leaving us?”
ML formulation	Predict Churn (0 = retained, 1 = churned) — a supervised binary classification problem.
Features (inputs)	Age, Gender, Income, PurchaseAmount, PurchaseFrequency, TenureMonths, SupportCalls, LastPurchaseDaysAgo.
Target (output)	Churn (binary label).
Evaluation metric	F1-score / ROC-AUC — more informative than raw accuracy when churners are a minority class (imbalanced data).
Business value	Earlier identification of at-risk customers enables targeted retention offers, reducing revenue loss.

4. Tensors

A **tensor** is a generalized, n-dimensional array of numbers — the core data structure used to represent data throughout machine learning and deep learning (NumPy arrays, TensorFlow/PyTorch tensors). A tensor is described by three properties:

Rank / ndim	The number of dimensions (axes) the tensor has.
Shape	The size of the tensor along each axis, e.g. (500, 8).
Dtype	The data type of its elements, e.g. float32, int64.

Tensors by Rank

Rank 0 — Scalar	A single number. Example: <code>np.array(42)</code> , shape <code>()</code> .
Rank 1 — Vector	A 1-D list of numbers, e.g. one customer's features <code>[Age, Income, PurchaseAmount]</code> , shape <code>(3,)</code> .
Rank 2 — Matrix	A 2-D grid — rows × columns. The full cleaned customer table is conceptually a rank-2 tensor, shape <code>(n_rows, n_cols)</code> .
Rank 3+ — N-D Tensor	Higher dimensional data, e.g. a batch of images with shape <code>(batch, height, width, channels)</code> .

NumPy Example (as used in the notebook)

```
import numpy as np

scalar    = np.array(42)                # rank 0, shape ()
vector    = np.array([34, 55000.0, 812.5]) # rank 1, shape (3,)
matrix     = np.array([[34, 55000.0, 812.5],
                       [28, 61000.0, 430.1]]) # rank 2, shape (2,3)
tensor3d   = np.stack([matrix, matrix * 1.05]) # rank 3, shape (2,2,3)

matrix.T.shape      # transpose -> (3, 2)
vector.reshape(3, 1) # reshape -> column vector
matrix.sum(axis=0)   # per-feature column totals
matrix.mean(axis=1)  # per-row (per-customer) mean
```

When a cleaned pandas DataFrame is passed into a machine learning model, it is converted into a numeric tensor of shape `(n_customers, n_features)` — which is exactly why cleaning, type-correction, and encoding (Sections 5 & 7) matter: every value in that tensor must be numeric.

5. Data Acquisition from Multiple Formats

Real organizations rarely keep all their data in one place. This project deliberately combines four common formats so the full range of import techniques is practiced:

CSV	Comma-Separated Values — a plain-text tabular format. Read with <code>pandas.read_csv()</code> . Common source: CRM exports.
JSON	JavaScript Object Notation — a nested key-value text format, common for web/app exports and API payloads. Parsed with the <code>json</code> module, then loaded into a <code>DataFrame</code> .
SQL Database	Structured, relational storage queried with SQL. Accessed in Python via a DB connector (e.g. <code>sqlite3</code>) and <code>pandas.read_sql_query()</code> .
REST API	A web service that returns data (typically JSON) over HTTP, fetched with the <code>requests</code> library. Production code should always include a fallback / retry path for network failures.

After loading, all four sources are combined with a common key (`CustomerID`) using an inner join, producing a single analytical dataset ready for cleaning.

6. Data Understanding & Cleaning

6.1 Initial Exploration

<code>.head()</code>	Displays the first few rows — a quick sanity check of structure and values.
<code>.info()</code>	Shows column dtypes, non-null counts, and memory usage — reveals type issues.
<code>.describe()</code>	Summary statistics (count, mean, std, min, quartiles, max) for a first read on distribution and possible outliers.

6.2 Common Data Quality Issues

- **Missing values** — cells with no recorded data (nulls/NaNs).
- **Duplicate rows** — the same record appearing more than once.
- **Inconsistent data types** — a numeric field stored as text (e.g. "\$54,200.00").
- **Inconsistent categorical labels** — the same category spelled differently ("Male", "M", "m").
- **Irrelevant / constant columns** — fields that carry no analytical value.
- **Outliers** — values far outside the typical range, which may be genuine or data-entry errors.

6.3 Data Cleaning Techniques

Imputation	Filling missing values using a statistic such as the median (robust to outliers), mean, or mode, or a model-based estimate.
Removal	Dropping rows/columns with missing data when imputation isn't appropriate (e.g. too much missingness, or the row can't be trusted).
Type correction	Casting columns to their correct dtype, e.g. stripping currency symbols before converting to float.
Standardizing categories	Mapping inconsistent labels to a single canonical set (e.g. all Gender values -> {Male, Female}).
Deduplication	Removing duplicate records, typically keeping the first or most recent occurrence.
Column pruning	Dropping columns that are constant, redundant, or irrelevant to the analysis/model.

7. Exploratory Data Analysis (EDA)

EDA is the practice of visually and statistically exploring a dataset to uncover patterns, spot anomalies, test hypotheses, and check assumptions before modeling.

Univariate analysis	Examines one variable at a time — e.g. a histogram of Age or Income to understand its distribution and shape (normal, skewed, bimodal).
Bivariate analysis	Examines the relationship between two variables — e.g. a boxplot of Income grouped by Churn to see if the two are related.
Multivariate analysis	Examines relationships among three or more variables simultaneously — e.g. a correlation heatmap or a pair plot colored by Churn.

Key visualizations used in this project include histograms with KDE overlays (univariate), boxplots (bivariate), and a correlation heatmap with a Seaborn pair plot (multivariate) — each paired with a short written interpretation, since a chart without interpretation provides little decision-making value.

8. Data Profiling

Data profiling is the automated process of examining a dataset to produce a structured summary of its metadata and quality: column types, value distributions, missing-value counts, cardinality, correlations, and data-quality warnings. Tools like `ydata-profiling` (formerly `pandas-profiling`) generate a full interactive HTML report with a single line of code:

```

from ydata_profiling import ProfileReport
profile = ProfileReport(df, title="Customer Data Profile", explorative=True)
profile.to_file("profiling_report.html")

```

A profiling report typically covers:

- **Overview** — number of rows/columns, duplicate rows, overall missing-cell percentage.
- **Variable summaries** — per-column statistics, most frequent values, distribution plots.
- **Correlations** — pairwise correlation matrix / heatmap across numeric variables.
- **Missing values** — count and pattern of missingness per column.
- **Warnings / alerts** — automatic flags such as high skewness, high correlation between two features, constant columns, or high cardinality categorical fields.

The accompanying notebook implements an equivalent, dependency-free profiling summary (so it runs without internet access), covering the same categories: overview, missing values, descriptive statistics, correlations, and automatic data-quality warnings.

9. Feature Engineering

Feature Engineering is the process of using domain knowledge to create, transform, or select variables ("features") that make patterns in the data more visible to a machine learning model. Good features often matter more to model performance than the choice of algorithm itself.

Encoding	Converting categorical variables into numeric form, e.g. one-hot or binary encoding of Gender, since ML models require numeric input.
Derived features / ratio	Combining existing columns into a new, more informative feature, e.g. AvgOrderValue = PurchaseAmount / PurchaseFrequency.
Binning	Converting a continuous variable into discrete groups, e.g. Age into AgeGroup buckets, which can help simpler models capture non-linear effects.
Scaling / normalization	Rescaling numeric features (e.g. with a StandardScaler) so that variables on different scales (Income vs. Age) contribute fairly to distance-based or gradient-based models.
Flag / indicator features	Boolean features that capture a business-meaningful condition, e.g. RecentlyActive = last purchase within 90 days.

In the notebook, these techniques are applied to the cleaned dataset to produce an **ML-ready feature matrix** — the final deliverable of the data preprocessing pipeline, ready to be handed to a classification algorithm for the churn-prediction task framed in Section 3.

10. Quick-Reference Glossary

Dataset	A structured collection of data, typically organized as rows (records) and columns (features/variables).
Feature	An individual measurable property or characteristic used as model input (a column in the dataset).
Target / Label	The variable a supervised model is trained to predict (e.g. Churn).
Missing value (NaN/null)	A cell in the dataset with no recorded value.
Outlier	A data point that differs significantly from other observations.
Skewness	A measure of the asymmetry of a distribution around its mean.
Correlation	A statistical measure (-1 to 1) of the linear relationship between two numeric variables.
Imputation	Replacing missing values with substituted, estimated values.
One-hot / binary encoding	Representing a categorical variable as one or more numeric (0/1) columns.
Standardization	Rescaling a numeric feature to have mean 0 and standard deviation 1.
Overfitting	When a model learns noise/specifics of training data and fails to generalize to new data.
Tensor	A generalized n-dimensional array of numbers; the core data structure in ML.